GHRAISONI
INTERNATIONAL
SKILL TECH UNIVERSITY

PUNE

Laboratory Manual

Object Oriented Programming Skills with C++ Lab (25BTPCP407)

Prepared by: Miss. Ashu Singh

G H RAISONI INTERNATIONAL SKILL TECH UNIVERSITY, PUNE

CERTIFICATE

This is certify that Mr. / Ms._____having roll no.: _____of B.Tech., Semester: _____, Registration no.: _____,Branch: _____, Division: _____ has satisfactorily completed the laboratory work of the subject: _____during the academic year: _____

Subject Teacher
School of Engineering and Technology,
GHRISTU, Pune

Head of Department
School of Engineering and Technology,
GHRISTU, Pune

Acknowledgement

It was a really good experience, working with **Object Oriented Programming Skills with C++ Lab.** First ,I would like to thank,

Dr. R. D. Kharadkar Vice-chancellor of G H Raisoni International Skill Tech University, Pune for his concern and giving technical support in fire forming the document.

Dr. R.D Kharadkar
(Vice-Chancellor)
G H Raisoni International Skill Tech University, Pune

GENERAL INSTRUCTIONS

1. Students are instructed to come to CPP laboratory on time. Late comers are not entertained in the lab.
2. Students should be punctual in the lab. If not, the assignments conducted will not be repeated.
3. Students are expected to come prepared at home with the assignments which are going to be performed.
4. Students are instructed to display their identity cards before entering the lab.
5. Students are instructed not to bring mobile phones to the lab.
6. Any damage/loss of system parts during the lab session, it is student's responsibility and penalty or fine will be collected from the student.
7. Students should update the records and lab observation books session wise.
8. Before leaving the lab, the student should get his lab observation book signed by the faculty.
9. Students should submit the lab records by the next lab to the concerned faculty members in the staffroom for their correction and return.
10. Students should not move around the lab during the lab session.
11. If any emergency arises, the student should take permission from faculty members concerned in written format.
12. The faculty members may suspend any student from the lab session on disciplinary grounds. Never copy the output from other students. Write down your own outputs

Guidelines to write your Observation book

1. Assignment title, Aim, code, input and output should be right side.
2. Model graphs, Observations table, Calculations table should be leftside.
3. Theoretical and model calculations can be any side as per convenience.
4. You must write the information for all the assignments in your observation
5. You are advised to leave sufficient no. of pages between successive assignments in your Lab book for model calculations.

Safety precautions

1. Students are supposed to come to the lab with preparation, proper dress code and their Laptops.
2. Disciplinary action can be taken in the event of mishandling the system or switching on the power supply without faculty presence.

Object Oriented Programming Skills with C++ Laboratory Assignments

1. Write a C++ program to demonstrate the concept of function overloading
2. Write a C++ program to pass object as a function argument.
3. Write a C++ program to demonstrate the following.
    a. static data member
    b. static member function
4. Write a C++ program to implement String Operations at least three.
5. Write a C++ program to implement friend function
6. Write a C++ program to exchange the private data member of two class
7. Write a C++ program to demonstrate the concept of function returning an object.
8. Write a C++ program to implement copy constructor.
9. Write a C++ program to implement the destructor
10. Write a C++ program to overload the binary '+' operator.
11. Write a C++ program to convert the class type data into primitive type data.
12. Write a C++ program to check whether a given number is prime or not?
13. Write a C++ program to find the largest value from the given
    a. Two numbers, b. Three numbers, c .Four numbers.
14. Write a C++ program to find sum of first 100 natural numbers by using
    a) for loop (b) while loop (c) do-while loop.
15. Write a C++ program to Bubble sort using template function.

Practical No. 1

Title: Write a C++ program to demonstrate the concept of function overloading.

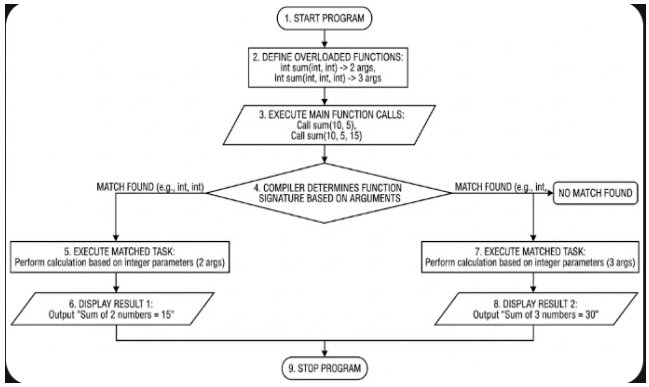
Aim: To implement multiple functions with the same name but different parameter lists.

Theory: Function overloading allows multiple functions with the same name but different signatures. The compiler differentiates them based on the number and type of arguments.

Algorithm:

1. Start the program.
2. Define multiple functions with the same name but different parameters.
3. Call each function to perform its task.
4. Display results.
5. Stop.

Flowchart:



Program:

```
#include <iostream>
using namespace std;
int sum(int a, int b) {
    return a + b;
}

int sum(int a, int b, int c) {
    return a + b + c;
}
```

```
int main() {
    cout << "Sum of 2 numbers = " << sum(10, 5) << endl;
    cout << "Sum of 3 numbers = " << sum(10, 5, 15) << endl;
    return 0;
}
```

Sample Output:

```
PROBLEMS    OUTPUT    SQL HISTORY    TERMINAL    ...    Filter (e.g. text, !excludeText, t...    Code
[Running] cd "c:\Users\HP\Desktop\C++\C++\Assignment\" && g++ Assignment1.cpp -o Assignment1 &&
"c:\Users\HP\Desktop\C++\C++\Assignment\"Assignment1
Sum of 2 numbers = 12
Sum of 3 numbers = 53

[Done] exited with code=0 in 6.691 seconds
```

Practical No. 2

Title: Write a C++ program to pass object as a function argument.
Aim: To demonstrate passing objects to a function.

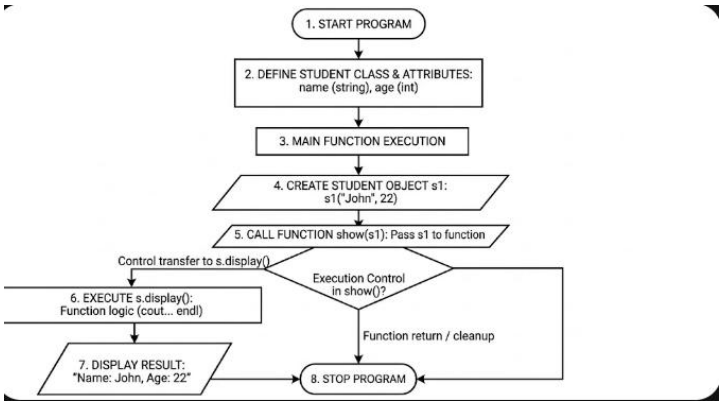
Theory: Objects can be passed to functions either by value, by reference, or by pointer.

This allows member data to be accessed or modified within the function.

Algorithm:

1. Start the program.
2. Define Class Student with a parameterized constructor and a display() member function.
3. Create Object s1 in the main() function with the values ("John", 22).
4. Call Global Function show(s1), which internally triggers the s1.display() method.
5. Display Output and Stop the program.

Flowchart



Program:

```
#include <iostream>
using namespace std;

class Student {
    string name;
    int age;
public:
    Student(string n, int a) : name(n), age(a) {}
```

```
    void display() {
        cout << "Name: " << name << ", Age: " << age << endl;
    }
};

void show(Student s) {
    s.display();
}

int main() {
    Student s1("John", 22);
    show(s1);
    return 0;
}
```

Sample Output:

```
PROBLEMS    OUTPUT    SQL HISTORY    TERMINAL    ...    Filter (e.g. text, !excludeText, t...    Code
[Running] cd "c:\Users\HP\Desktop\C++\C++\Assignment\" && g++ Assignment2.cpp -o Assignment2 &&
"c:\Users\HP\Desktop\C++\C++\Assignment\"Assignment2
Name: Prem, Age: 19

[Done] exited with code=0 in 5.962 seconds
```

Practical No. 3

Title: Write a C++ program to demonstrate static data member and static member function.

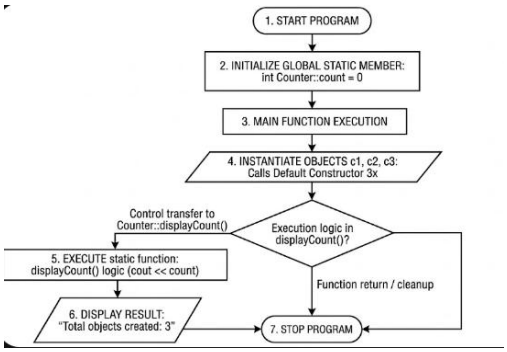
Aim: To show how static data members and static member functions work in C++.

Theory: Static members belong to the class rather than any specific object. They are shared by all objects of the class.

Algorithm:

1. Start the program.
2. Define Class Counter with a static integer count (initialized to 0) and a default constructor that increments count.
3. Define Static Member Function displayCount() within the class to print the current value of count.
4. In main() function, Instantiate Objects c1, c2, c3, which automatically calls the default constructor three times, incrementing count from 0 to 3.
5. Call Counter::displayCount(), which displays "Total objects created: 3", then Stop the program.

Flowchart:



Program:

```
#include <iostream>
using namespace std;
class Counter {
    static int count;
public:
    Counter() { count++; }
    static void displayCount() {
        cout << "Total objects created: " << count << endl;
```

```
    }
};
int Counter::count = 0;
int main() {
    Counter c1, c2, c3;
    Counter::displayCount();
    return 0;
}
```

Sample Output:

```
[Running] cd "c:\Users\HP\Desktop\C++\C++\Assignment\" && g++ Assignment3.cpp -o Assignment3 &&
"c:\Users\HP\Desktop\C++\C++\Assignment\"Assignment3
Total objects created: 4

[Done] exited with code=0 in 1.051 seconds
```

Practical No. 4

Title: Write a C++ program to implement String Operations (concatenation, reverse, length).

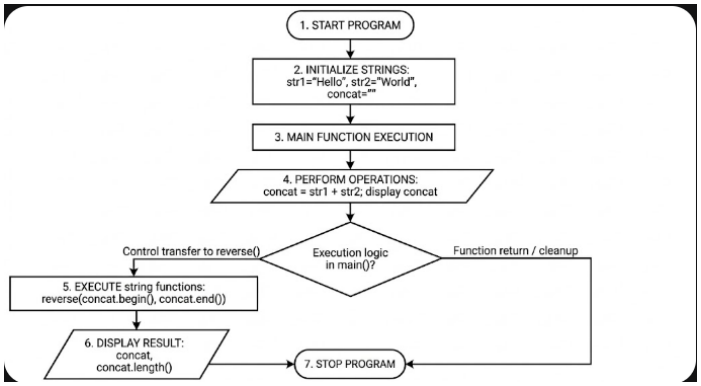
Aim: To perform basic string operations using the C++ string class.

Theory: String operations can be implemented using built-in string functions and loops.

Algorithm:

1. Start the program.
2. Initialize two strings, `str1` ("Hello") and `str2` ("World").
3. Concatenate `str1` and `str2` into a new string `concat` and display it.
4. Reverse the `concat` string using the `reverse()` function.
5. Display the reversed string and its length, then Stop.

Flowchart:



Program:

```
#include <iostream>
#include <algorithm>
using namespace std;
int main() {
    string str1 = "Hello", str2 = "World";
    string concat = str1 + str2;
    cout << "Concatenated String: " << concat << endl;
    reverse(concat.begin(), concat.end());
    cout << "Reversed String: " << concat << endl;
```

```
    cout << "Length: " << concat.length() << endl;
    return 0;
}
```

Sample Output:

```
[Running] cd "c:\Users\HP\Desktop\C++\C++\Assignment\" && g++ Assignment4.cpp -o Assignment4 &&
"c:\Users\HP\Desktop\C++\C++\Assignment\"Assignment4
Concatenated String: PremDhone
Reversed String: enohDmerP
Length: 9

[Done] exited with code=0 in 3.411 seconds
```

Practical No. 5

Title: Write a C++ program to implement a friend function.

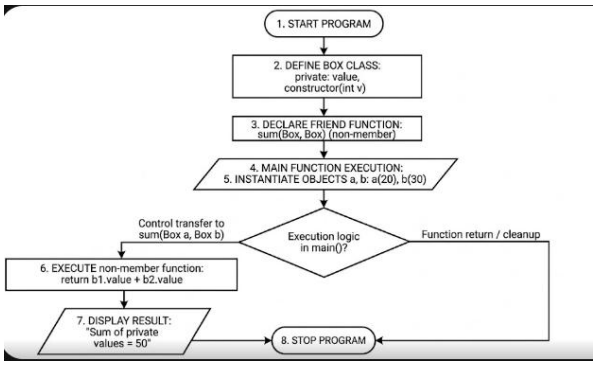
Aim: To use friend function for accessing private members.

Theory: Friend functions can access private members of a class.

Algorithm:

1. Start the program.
2. Define Class `Box`: Declare a private integer `value` and a parameterized constructor to initialize it.
3. Declare `sum()` as Friend: Inside the `Box` class, declare the `sum(Box, Box)` function as a `friend`.
4. Define `sum()` Function: Outside the class, define `sum()` to access the private `value` of two `Box` objects and return their sum.
5. Enter `main()`: Instantiate objects `a(20)` and `b(30)`, call `sum(a, b)`, display the result (50), and Stop.

Flowchart:



Program:

```
#include <iostream>
using namespace std;
class Box {
    int value;
public:
    Box(int v) : value(v) {}
    friend int sum(Box, Box);
};
```

```
int sum(Box b1, Box b2) {
    return b1.value + b2.value;
}
int main() {
    Box a(20), b(30);
    cout << "Sum of private values = " << sum(a, b) << endl;
}
```

Sample Output:

```
[Running] cd "c:\Users\HP\Desktop\C++\C++\Assignment\" && g++ Assignment5.cpp -o Assignment5 &&
"c:\Users\HP\Desktop\C++\C++\Assignment\"Assignment5
Sum of private values = 80

[Done] exited with code=0 in 1.128 seconds
```

Practical No. 6

Title: Write a C++ program to exchange the private data members of two classes.

Aim: To swap data between objects of different classes using friend function.

Theory: Friend function can access private members of multiple classes.

Algorithm:

- Start
- Create object `A = 10` and `B = 20`
- Call `exchange (A, B)`
- Swap values using temp variable
- Display values of A and B
- End

Flowchart:

```
 Start
   |
   ▼
Create A=10, B=20
   |
   ▼
Call exchange(A,B)
   |
   ▼
Swap values
   |
   ▼
Print A, B
   |
   ▼
 End
```

Program:

```
#include <iostream>
using namespace std;
class B;
class A {
   int x;
public:
   A(int v) : x(v) {}
```

```
    friend void exchange(A&, B&);
    void show() { cout << "A: " << x << endl; }
};
class B {
    int y;
public:
    B(int v) : y(v) {}
    friend void exchange(A&, B&);
```

```
    void show() { cout << "B: " << y << endl; }
};
void exchange(A &obj1, B &obj2) {
    int temp = obj1.x;
    obj1.x = obj2.y;
    obj2.y = temp;
}
int main() {
    A a(10); B b(20);
    exchange(a, b);
    a.show();
    b.show();
}
```

Sample Output:

```
PS C:\Users\HP\Desktop\C++\C++> cd "c:\Users\HP\De
pp -o Assignment6 } ; if ($?) { .\Assignment6 }
A: 20
B: 10
PS C:\Users\HP\Desktop\C++\C++\Assignment>
```

Practical No. 7

Title: Write a C++ program to demonstrate the concept of function returning an object.

Aim: To return objects from functions.

Theory: Objects can be returned from functions like any other data type.

Algorithm:

1. Start
2. Create objects `n1 = 10`, `n2 = 32`
3. Call `n3 = n1.add(n2)`
4. Add values: `n3 = n1 + n2`
5. Display result using `n3.show()`
6. End

Flowchart

```
 Start
   |
   ▼
Create n1=10, n2=32
   |
   ▼
Call n3 = n1.add(n2)
   |
   ▼
Add values
   |
   ▼
Print result
   |
   ▼
 End
```

Program:

```
#include <iostream>
using namespace std;
class Number {
    int x;
public:
    Number(int a=0) : x(a) {}
    void show() { cout << "Value: " << x << endl; }
    Number add(Number n) {
        return Number(x + n.x);
    }
};
```

```
int main() {
    Number n1(10), n2(32);
    Number n3 = n1.add(n2);
    n3.show();
}
```

Sample Output:

```
PS C:\Users\HP\Desktop\C++\C++\Assignment> cd "c:\Users\HP\
signment7.cpp -o Assignment7 } ; if ($?) { .\Assignment7 }
Value: 82
PS C:\Users\HP\Desktop\C++\C++\Assignment>
```

Practical No. 8

Title: Write a C++ program to implement a copy constructor.

Aim: To create a new object as a copy of an existing object.

Theory: Copy constructor initializes an object using another object.

Algorithm:

1. Start
2. Create object `s1 = 25`
3. Create `s2` using copy constructor (`s2 = s1`)
4. Copy value of `s1` to `s2`
5. Display value using `s2.show()`
6. End

Flowchart

```
 Start
   |
   ▼
Create s1 = 25
   |
   ▼
Copy s1 → s2
   |
   ▼
Print s2 value
   |
   ▼
 End
```

Program:

```
#include <iostream>
using namespace std;
class Sample {
    int x;
public:
    Sample(int a) : x(a) {}
    Sample(Sample &obj) { x = obj.x; }
    void show() { cout << "Value: " << x << endl; }
};
```

```
int main() {
    Sample s1(25);
    Sample s2(s1);
    s2.show();
}
```

Sample Output:

```
PS C:\Users\HP\Desktop\C++\C++\Assignment> cd "c:\Users\HP\
signment8.cpp -o Assignment8 } ; if ($?) { .\Assignment8 }
Value: 76
PS C:\Users\HP\Desktop\C++\C++\Assignment>
```

Practical No. 9

Title: Write a C++ program to implement a destructor.

Aim: To destroy objects when they go out of scope.

Theory: Destructor is called automatically to release resources.

Algorithm:

1. Start
2. Create object `d1 = 1` (Constructor called)
3. Display constructor message
4. Program ends → Destructor called
5. Display destructor message
6. End

Flowchart

```
 Start
   |
   ▼
Create d1 = 1
(Constructor called)
   |
   ▼
Display message
   |
   ▼
Program ends
   |
   ▼
Destructor called
   |
   ▼
 End
```

Program:

```
#include <iostream>
using namespace std;
class Demo {
   int id;
public:
   Demo(int i) : id(i) { cout << "Constructor " << id << endl; }
   ~Demo() { cout << "Destructor called for object ID: " << id <<
endl; }
};
int main() {
   Demo d1(1);
}
```

Sample Output:

```
signment9.cpp -o Assignment9 } ; if ($?) { .\Assignment9 }
Constructor 1
Destructor called for object ID: 1
PS C:\Users\HP\Desktop\C++\C++\Assignment>
```

Practical No. 10

Title: Write a C++ program to overload the binary '+' operator.

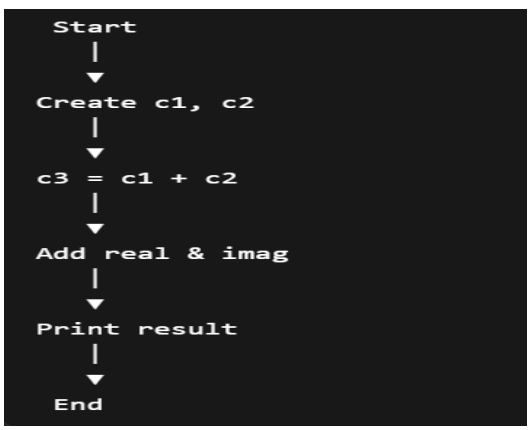
Aim: To add two complex numbers using operator overloading.

Theory: Operator functions can be defined for user-defined types.

Algorithm:

1. Start
2. Create objects `c1 = (2,3)` and `c2 = (3,4)`
3. Add using operator overloading: `c3 = c1 + c2`
4. Add real and imaginary parts
5. Display result using `c3.display()`
6. End

Flowchart



Program:

```
#include <iostream>
using namespace std;
class Complex {
    int real, imag;
public:
    Complex(int r=0, int i=0) : real(r), imag(i) {}
    Complex operator + (Complex const &obj) {
        return Complex(real + obj.real, imag + obj.imag);
    }
    void display() { cout << real << " + " << imag << "i" << endl; }
};
```

```
int main() {
    Complex c1(2,3), c2(3,4);
    Complex c3 = c1 + c2;
    c3.display();
}
```

Sample Output:

```
PS C:\Users\HP\Desktop\C++\C++\Assignment> cd "c:\Users\HP\Desk
signment10.cpp -o Assignment10 } ; if ($?) { .\Assignment10 }
11 + 8i
PS C:\Users\HP\Desktop\C++\C++\Assignment>
```

Practical No. 11

Title: Write a C++ program to convert class type to primitive type.

Aim: To define conversion functions.

Theory: Conversion functions convert object data to primitive type.

Algorithm:

1. Start
2. Create object `num = 45`
3. Convert object to integer (`val = num`)
4. Store value in `val`
5. Display `val`
6. End

Flowchart

```
 Start
   |
   ▼
Create num = 45
   |
   ▼
Convert num → int
   |
   ▼
Store in val
   |
   ▼
Print val
   |
   ▼
 End
```

Program:

```
#include <iostream>
using namespace std;
class Number {
    int x;
public:
    Number(int a) : x(a) {}
    operator int() { return x; }
};
int main() {
```

```
    Number num(45);
    int val = num;
    cout << "Converted value = " << val << endl;
}
```

Sample Output:

```
PS C:\Users\HP\Desktop\C++\C++\Assignment> cd "c:\Users\HP\
signment11.cpp -o Assignment11 } ; if ($?) { .\Assignment11
Converted value = 18
PS C:\Users\HP\Desktop\C++\C++\Assignment>
```

Practical No. 12

Title: Write a C++ program to check whether a given number is prime or not.

Aim: To use loops for prime number check.

Theory: Prime number has only two factors: 1 and itself.

Algorithm:

1. Start
2. Initialize `a=5, b=9, c=2, d=78`
3. Find max of `a` and `b`
4. Compare with `c` → update max
5. Compare with `d` → update max
6. Display largest value
7. End

Flowchart

```
 Start
   |
   ▼
Initialize a,b,c,d
   |
   ▼
max = (a>b)?a:b
   |
   ▼
Compare with c
   |
   ▼
Compare with d
   |
   ▼
Print max
   |
   ▼
 End
```

Program:

```
#include <iostream>
using namespace std;
int main() {
    int n = 13, i;
    bool prime = true;
    for(i=2; i<=n/2; i++) {

        if(n % i == 0) { prime = false; break; }
```

```
    }
    if(prime) cout << n << " is a prime number";
    else cout << n << " is not a prime number";
}
```

Sample Output:

```
signment12.cpp -o Assignment12 } ; if ($?) { .\Assignment12 }
17 is a prime number
PS C:\Users\HP\Desktop\C++\C++\Assignment>
```

Practical No. 13

Title: Write a C++ program to find the largest value from 2, 3, and 4 numbers.

Aim: To use conditional statements to find maximum.

Theory: Comparisons can be done using if-else.

Algorithm:

1. Start
2. Initialize `a=5, b=9, c=2, d=78`
3. Find maximum using conditional operator
4. Store result in `max`
5. Display `max`
6. End

Flowchart

```
 Start
   |
   ▼
Initialize a,b,c,d
   |
   ▼
Find max using ?:
   |
   ▼
Store in max
   |
   ▼
Print max
   |
   ▼
 End
```

Program:

```
#include <iostream>
using namespace std;
int main() {
    int a=5,b=9,c=2,d=78;
    int max = (a>b)?a:b;
    max = (max>c)?max:c;
    max = (max>d)?max:d;
    cout << "Largest = " << max;
```

```
}
```

Sample Output:

```
signment13.cpp -o Assignment13 } ; if ($?) { .\Assignment13 }
Largest = 97
PS C:\Users\HP\Desktop\C++\C++\Assignment>
                                                          Ln
```

Practical No. 14

Title: Write a C++ program to find sum of first 100 natural numbers using loops.

Aim: To use for, while, and do-while loops.

Theory: Loops can iterate to sum numbers.

Algorithm:

1. Start
2. Initialize `sum = 0`
3. Loop from `i = 1` to `100`
4. Add `i` to `sum`
5. Display `sum`
6. End

Flowchart

```
 Start
   |
   ▼
Initialize sum=0
   |
   ▼
i = 1
   |
   ▼
i ≤ 100 ?
  ┌────┴────┐
  |  Yes    |
 ▼         | No
sum=sum+i|
 |        ▼
 ▼     Print sum
i=i+1      |
 |         ▼
 └──────── End
```

Program:

```
#include <iostream>
using namespace std;
int main() {
   int sum=0;
   for(int i=1;i<=100;i++) sum+=i;
   cout << "Sum = " << sum;
}
```

Sample Output:

```
PS C:\Users\HP\Desktop\C++\C++\Assignment> cd "c:\Users\HP\Desk
signment14.cpp -o Assignment14 } ; if ($?) { .\Assignment14 }
Sum = 55
```

Practical No. 15

Title: Write a C++ program to Bubble sort using template function.

Aim: To sort elements using Bubble sort algorithm.

Theory: Templates make function generic.

Algorithm:

1. Start
2. Initialize array `{9,2,7,12,5}`
3. Apply **bubble sort**:
4. Compare adjacent elements
5. Swap if needed
6. Repeat until array is sorted
7. Print sorted array
8. End

Flowchart:

```
 Start
   |
   ▼
Initialize array
   |
   ▼
Repeat passes
   |
   ▼
Compare adj elements
   |
   ▼
Swap if needed
   |
   ▼
Array sorted?
 ┌────┴────┐
 |  No     |
 ▼         | Yes
Repeat     ▼
        Print array
            |
            ▼
           End
```

Program:

```
#include <iostream>
using namespace std;
template <class T>
void bubbleSort(T arr[], int n) {
   for(int i=0;i<n-1;i++)
      for(int j=0;j<n-i-1;j++)
         if(arr[j] > arr[j+1])
            swap(arr[j], arr[j+1]);
}
int main() {
   int arr[5] = {9,2,7,12,5};
   bubbleSort(arr,5);
   for(int i=0;i<5;i++) cout << arr[i] << " ";
}
```

Sample Output:

```
signment15.cpp -o Assignment15 } ; if ($?) { .\Assignment15 }
2 3 5 7 8
PS C:\Users\HP\Desktop\C++\C++\Assignment>
```